

Building reusable bioimaging workflows with Nextflow

Sebastian Gonzalez Tirado

Shout-outs

- James Fellow Yates (Leibniz-HKI / MPI-EVA)
- Christian Tischer (EMBL)

Outcomes of this talk

- What are workflow managers and why should I care?
- Examples of complex bioimaging workflows orchestrated with Nextflow.
- Practical recommendations to create your own workflows.

Have you ever faced one of these problems?

Discovering an exciting new tool

S: “Oh cool, let’s go to the repo and check it out!”



Cell segmentation in imaging-based spatial transcriptomics

Viktor Petukhov^{1,2}, Rosalind J. Xu^{1,2,3}, Ruslan A. Soldatov¹, Paolo Cadini^{1,2}, Konstantin Khodosevich^{1,2}, Jeffrey R. Moffitt^{1,2,3} and Peter V. Kharchenko^{1,2,3}

...could not even install it.

Have you ever faced one of these problems?

Discovering an exciting new tool

S: “Oh cool, let’s go to the repo and check it out!”



Cell segmentation in imaging-based spatial transcriptomics

Viktor Petukhov^{1,2}, Rosalind J. Xu^{1,2,3}, Ruslan A. Soldatov¹, Paolo Cadini^{1,2}, Konstantin Khodosevich^{1,2}, Jeffrey B. Moffitt^{1,2,3} and Peter V. Kharchenko^{1,2,3}

...could not even install it.

Repetitive use of tools



**track
mate**

Have you ever faced one of these problems?

Discovering an exciting new tool

S: “Oh cool, let’s go to the repo and check it out!”



Cell segmentation in imaging-based spatial transcriptomics

Viktor Petukhov^{1,2}, Rosalind J. Xu^{1,2}, Ruslan A. Soldatov¹, Paolo Cadini^{1,2}, Konstantin Khodosevich^{1,2}, Jeffrey R. Moffitt^{1,2} and Peter V. Kharchenko^{1,2}

...could not even install it.

nature
biotechnology

ARTICLES

<https://doi.org/10.1038/s41587-020-0044-4>

Open Access

Repetitive use of tools



track
mate

Make sure your work is reusable

h-index from 0 to hero

Have you ever faced one of these problems?

Discovering an exciting new tool

S: “Oh cool, let’s go to the repo and check it out!”



...could not even install it.

Repetitive use of tools



track
mate

Make sure your work is reusable

h-index from 0 to hero

Adapting slowly into standards

nf-core



Only one rule for bioimage analysts club: be lazy

What are workflow managers?

Like orchestra directors managing the flow of your pipeline/symphony.

What do they do?

Workflow managers provide a framework for the creation, execution, and monitoring of a pipeline. <...>

They simplify pipeline development, optimize resource usage, handle software installation and versions, and run on different compute platforms, enabling workflow portability and sharing.

Wratten et al. (2021) Nature Methods

Other benefits

You also benefit from

- Increased **portability**
- More **efficiency**:
 - Less headaches from installation
 - Better resource management and monitoring
 - Extremely parallelised

Other benefits

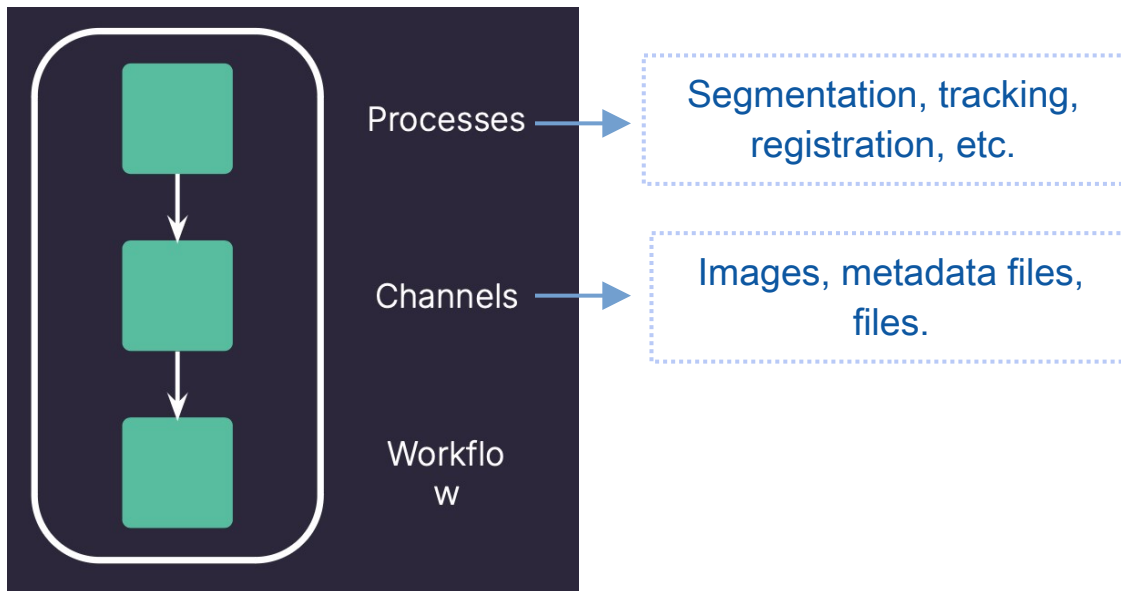
You also benefit from

REUSABLE

- Increased **portability**
- More **efficiency**:
 - Less headaches from installation
 - Better resource management and monitoring
 - Extremely parallelised

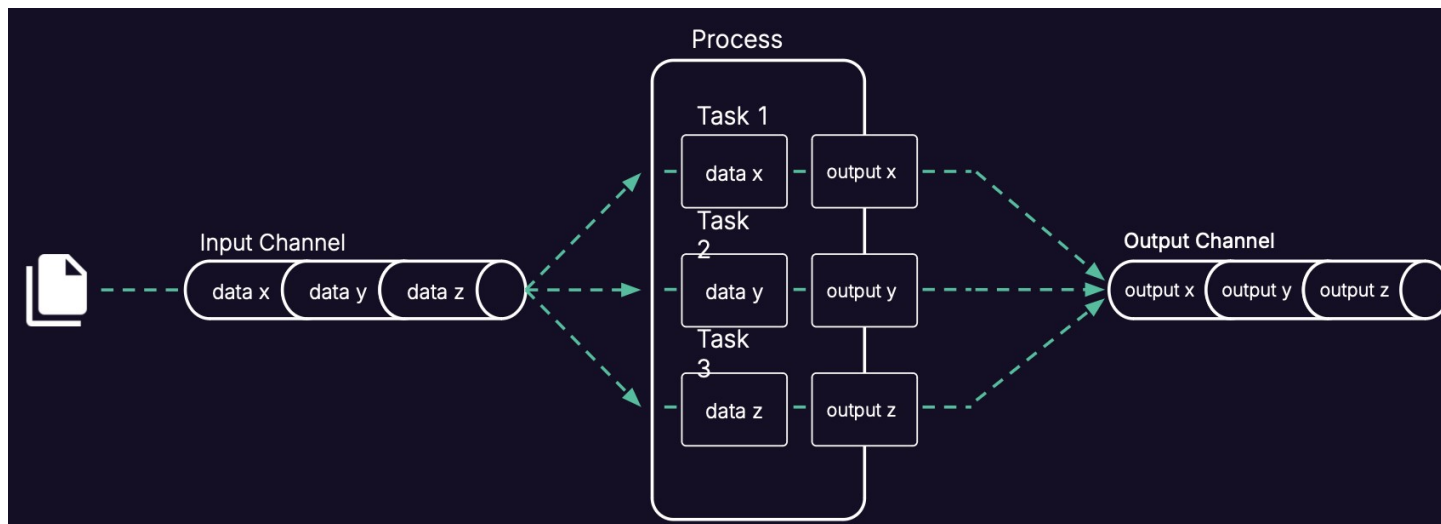
Getting more technical

Nextflow is a workflow orchestration engine and domain-specific language (DSL) that makes it easy to write data-intensive computational workflows. **It is designed around the idea that the Linux platform is the lingua franca of data science.**



Getting more technical

Nextflow is a workflow orchestration engine and domain-specific language (DSL) that makes it easy to write data-intensive computational workflows. **It is designed around the idea that the Linux platform is the lingua franca of data science.**



In line with our first rule: do not reinvent the wheel

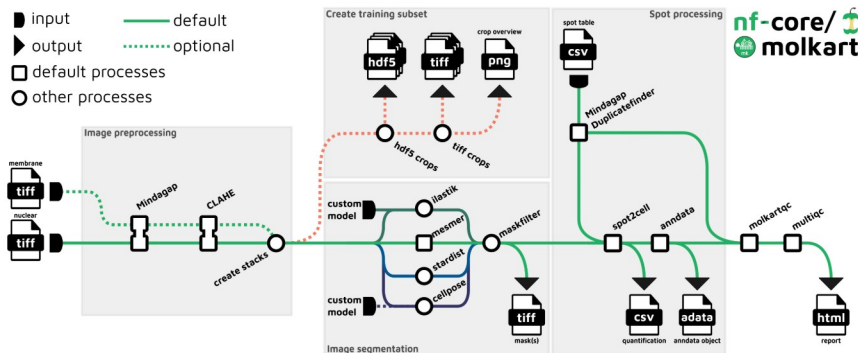
nf-core



A **community** effort to collect a curated set of analysis pipelines built using  Nextflow.

Introduction

nf-core/molkart is a pipeline for processing Molecular Cartography data from Resolve Bioscience (combinatorial FISH). It takes as input a table of FISH spot positions (x,y,z,gene), a corresponding DAPI image (TIFF format) and optionally an additional staining image in the TIFF format. nf-core/molkart performs end-to-end processing of the data including image processing, QC filtering of spots, cell segmentation, spot-to-cell assignment and reports quality metrics such as the spot assignment rate, average spots per cell and segmentation mask size ranges.



>_ run with

nf-core pipelines launch nf-cor

nf-core	Nextflow	Sequera Platform
---------	----------	------------------

subscribers	stars
163	13
open issues	open PRs
11	2
last release	last update
8 months ago	8 months ago

included modules

cellpose deepcell_mesmer ilastik_multicut
ilastik_pixelclassification
mindagap_duplicatefinder and 3 more modules

included subworkflows

utils_nextflow_pipeline utils_nfcore_pipeline
utils_nfschema_plugin

contributors



Adapting to nf-core specs takes time

You can also just write your own pipeline:

<https://github.com/BiolImageTools/iss-nf>

iss-nf: A Nextflow-based end-to-end *in situ* sequencing decoding workflow

 Nima Vakili, Sebastián González-Tirado,  Nils Kurzawa,  Dmytro Dvornikov, Zeinab Mokhtari, Frank Wippich,  Giovanna Bergamini,  Rainer Pepperkok,  Christian Tischer,  Luis A. Vale-Silva

doi: <https://doi.org/10.1101/2025.10.16.682795>

This article is a preprint and has not been certified by peer review [what does this mean?].

Main components of a nextflow script (ex: **test.nf**)

Channels definition

```
images_ch = Channel
    .fromPath("${params.inputDir}/*C.tif")
```

Workflow sequence

```
workflow = {
    labels = SEGMENT ( images )
    MEASURE ( images.join( labels ) )
}
```

Define processes

```
process SEGMENT {
    input:
    path(image)
    output:
    path("*.tif")

    script:
    """
    cellpose --input ${image} --pretrained_model cyto --save_tif
    """
}

process MEASURE {
    publishDir '${params.outputDir}', mode: 'copy'
    input:
    tuple path(image), path(labels)
    output:
    path("*.csv")

    script:
    """
    fiji --run measure.groovy "${image}, ${labels}, ${imageID}.csv"
    """
}
```


How do we run it?

Just type from the terminal:

```
$ nextflow run your_nextflow_script.nf --inputDir /path/2/images
```

```
N E X T F L O W ~ version 24.10.4
```

```
Launching `practice_nextflow.nf` [goofy_liskov] DSL2 - revision: a7c13f3288
```

```
executor > local (8)
```

```
[ad/a2a962] SEGMENT_CELLPOSE (2) [100%] 4 of 4 ✓
```

```
[7e/d16a1e] SEGMENT_INSTANSEG (4) [100%] 4 of 4 ✓
```

```
Completed at: 26-Oct-2025 09:15:25
```

```
Duration : 4m 58s
```

```
CPU hours : 0.3
```

```
Succeeded : 8
```

Channels: factories and operators

We create channels using “**factories**”. Basically, factories specify what type of channel to create and from which type of object:

Contents of test.nf

```
// Creating channels from channel factories:
images_ch = Channel.fromPath("${params.inputDir}/*C.tif")
metadata_ch = Channel.fromPath("${params.inputDir}/*_metadata.json")
channel.fromList( ['a', 'b', 'c', 'd'] )
```

Channels: factories and operators

Operators: methods that help to manipulate your channels:

Contents of test.nf

```
// Basic print operator on a channel:  
test_ch = Channel.from('one', 'two', 'three')  
test_ch.view()
```

Running script and output

```
> nextflow run print_channel.nf  
Nextflow 25.10.0 is available - Please consider updating your version to it  
  
NEXTFLOW ~ version 24.10.4  
  
Launching `print_channel.nf` [hopeful_miescher] DSL2 - revision: 41e48b8336  
  
one  
two  
three
```

Some of the most important aspects to consider

Configuration files (my_file.config):

```
process {  
  withName:SEGMENT_CELLPOSE {  
    conda = '/Users/sebgoti/miniforge3/envs/cellpose'  
  }  
  
  withName: SEGMENT_INSTANSEG {  
    conda = '/Users/sebgoti/miniforge3/envs/instanseghenv'  
  }  
}  
  
conda.enabled = true  
docker.enabled = false  
  
report {  
  enabled = true  
  //file = "${params.outdir}/reports/execution_report.html"  
  overwrite = true  
}  
  
params {  
  inputDir = "/Users/sebgoti/Documents/PhD/Riken_Globias/GloBIAS"  
}
```

Some of the most important aspects to consider

Configuration files (my_file.config):

```
process {
  withName: SEGMENT_CELLPOSE {
    conda = '/Users/sebgoti/miniforge3/envs/cellpose'
  }

  withName: SEGMENT_INSTANSEG {
    conda = '/Users/sebgoti/miniforge3/envs/instanseghenv'
  }
}

conda.enabled = true
docker.enabled = false

report {
  enabled = true
  //file = "${params.outdir}/reports/execution_report.html"
  overwrite = true
}

params {
  inputDir = "/Users/sebgoti/Documents/PhD/Riken_Globias/GloBIAS"
}
```

```
process {
  executor = 'slurm'
  queue = 'htc-el8'
  cpus = 8
  memory = 16.GB
  time = 20.min
  container = "docker://segonzal/fish_analysis:0.0.3"

  withLabel: 'long' {
    cpus = 8
    memory = 32.GB
    container = "docker://segonzal/fish_analysis:0.0.3"
  }

  withLabel: 'registration' {
    cpus = 4
    memory = 32.GB
  }

  withLabel: 'min' {
    cpus = 1
    memory = 400.MB
  }
}
```

Some of the most important aspects to consider

Output files after running a nextflow script:

```
N E X T F L O W ~ version 24.10.4

Launching `practice_nextflow.nf` [goofy_liskov] DSL2 - revision: a7c13f3288

executor > local (8)
[ad/a2a962] SEGMENT_CELLPOSE (2) [100%] 4 of 4 ✓
[7e/d16a1e] SEGMENT_INSTANSEG (4) [100%] 4 of 4 ✓
Completed at: 26-Oct-2025 09:15:25
Duration      : 4m 58s
CPU hours     : 0.3
Succeeded     : 8
```

Hash where output is
saved inside **/work**
directory



That is it for the theory!

Before moving on to the exercises, questions???

Important references:

- Nextflow documentation:

<https://www.nextflow.io/docs/latest/index.html>

- Nf-core:

<https://nf-co.re/>

- Official training resources:

<https://training.nextflow.io/2.0/>

Familiarize yourself with it

Three step exercise:

A. Create channel containing 4 tif images, located in
‘/home/GloBIAS_2025/data4nextflow’

and use the `.view()` operator to print them.

B. Create process that copies images with the ***_copy.tif** suffix:

→ Do not forget to include a workflow section with the process!!!

```
process COPY {  
  input:  
    path(image)  
  output:  
    path(*_copy.tif)  
  
  script:  
    """  
    cp ${image} ${image.baseName}_copy.tif  
    """  
}
```


Familiarize yourself with it

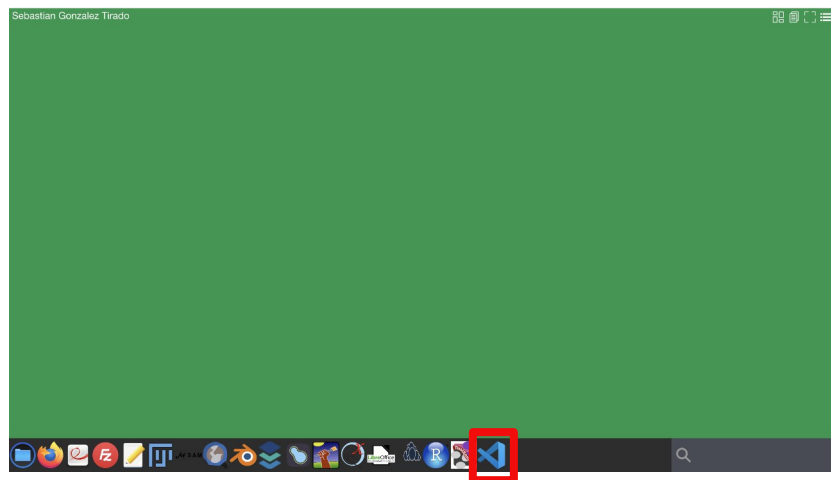
Three step exercise:

C. (Optional and any team to do it wins a price) Create a process that segments the images using CellposeSAM. Bash command could look like this:

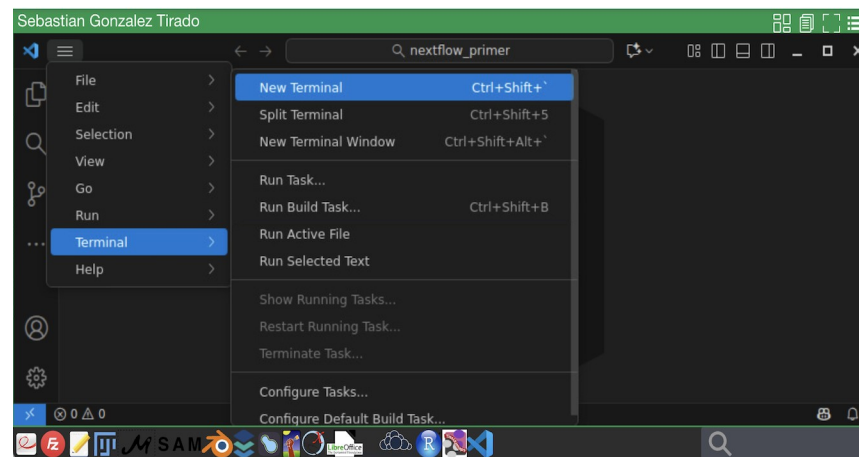
```
$ cellpose --image_path ${image} --save_tif
```

Step-by-step guide

1. Make teams and access BARD (with GPU).
2. Open VSCode and launch a Terminal inside it:

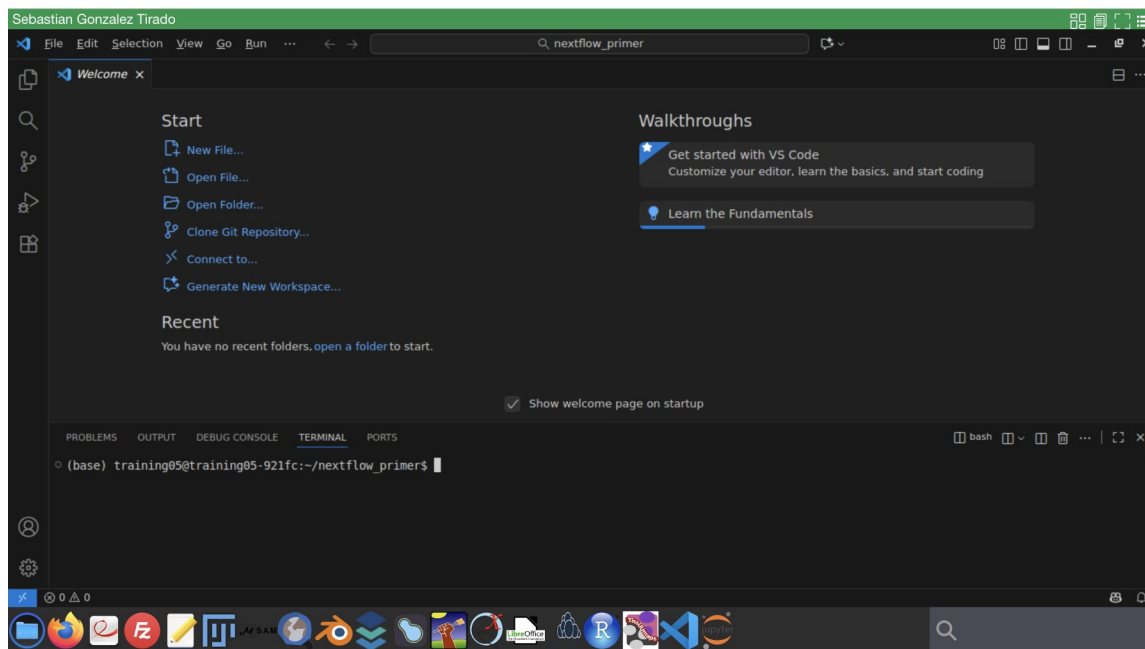


Click VSCode



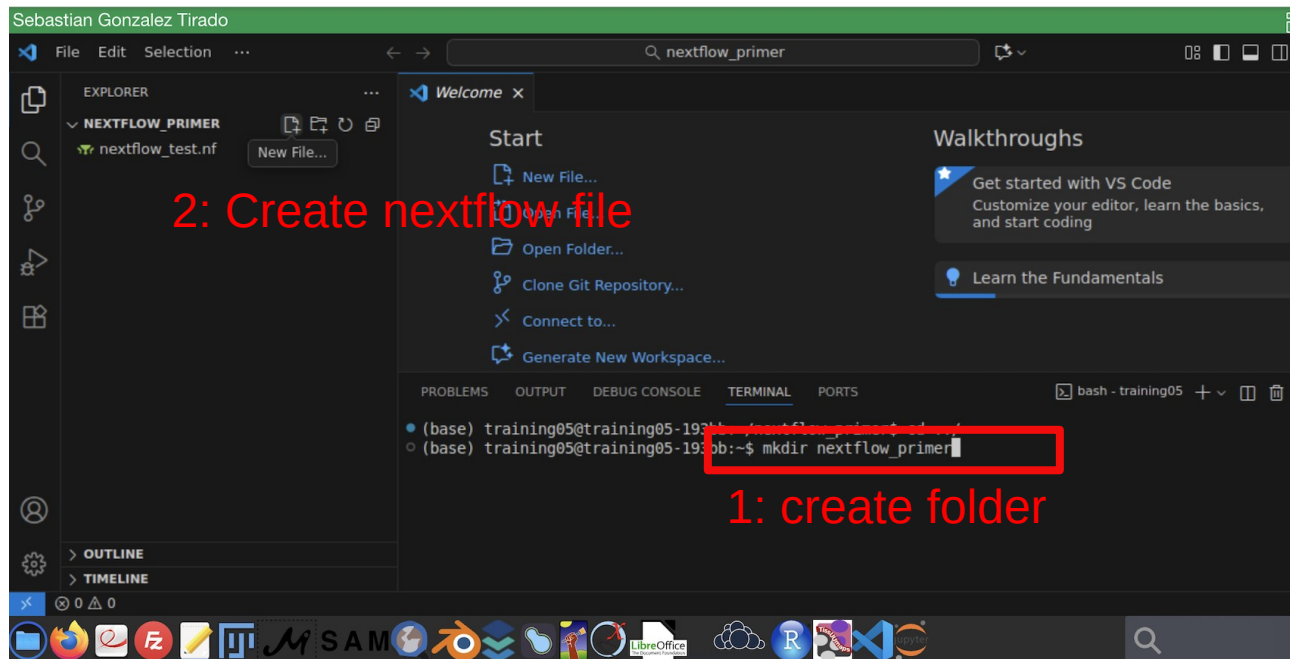
Step-by-step guide

Your VSCode should look like this:



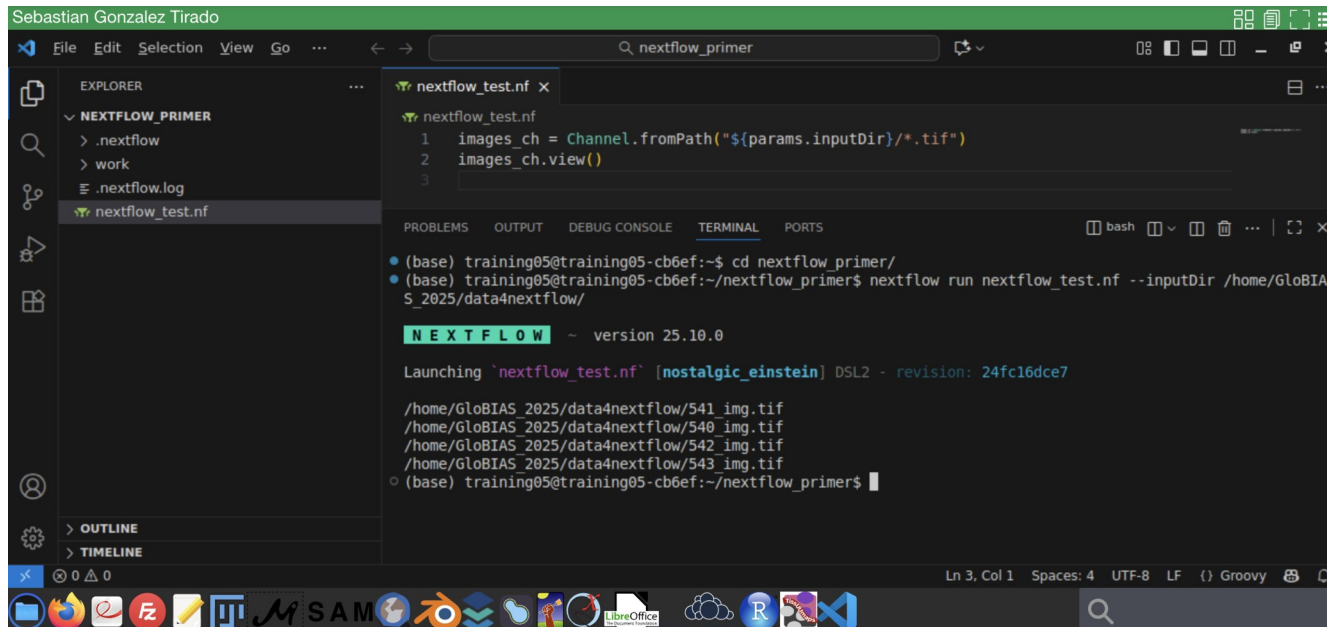
Step-by-step guide

3. Create a directory named “nextflow_primer” and then a file named ‘nextflow_test.nf’ inside it. Open the the file in VSCode:



Familiarize yourself with it

4. Open the file, create the channel and print the tif files by running the nextflow script:



The screenshot shows an IDE window titled "Sebastian Gonzalez Tirado" with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named "NEXTFLOW_PRIMER" with files ".nextflow", "work", ".nextflow.log", and "nextflow_test.nf". The "nextflow_test.nf" file is open in the editor, showing the following code:

```

nextflow_test.nf
1  images_ch = Channel.fromPath("${params.inputDir}/*.tif")
2  images_ch.view()
3

```

The terminal output shows the execution of the script:

```

(base) training05@training05-cb6ef:~$ cd nextflow_primer/
(base) training05@training05-cb6ef:~/nextflow_primer$ nextflow run nextflow_test.nf --inputDir /home/GloBIAS_2025/data4nextflow/

NEXTFLOW ~ version 25.10.0

Launching 'nextflow_test.nf' [nostalgic_einstein] DSL2 - revision: 24fc16dce7

/home/GloBIAS_2025/data4nextflow/541_img.tif
/home/GloBIAS_2025/data4nextflow/540_img.tif
/home/GloBIAS_2025/data4nextflow/542_img.tif
/home/GloBIAS_2025/data4nextflow/543_img.tif
(base) training05@training05-cb6ef:~/nextflow_primer$

```

Familiarize yourself with it

5. Create the COPY process and do part B (no more help from this point).

6. Create a CELLPOSE process using the command given in C. Note: you will need to activate conda and a cellpose environment with the following commands from the terminal inside VSCode:

```
$ source /opt/conda/etc/profile.d/conda.sh
```

```
$ conda activate cellpose4
```